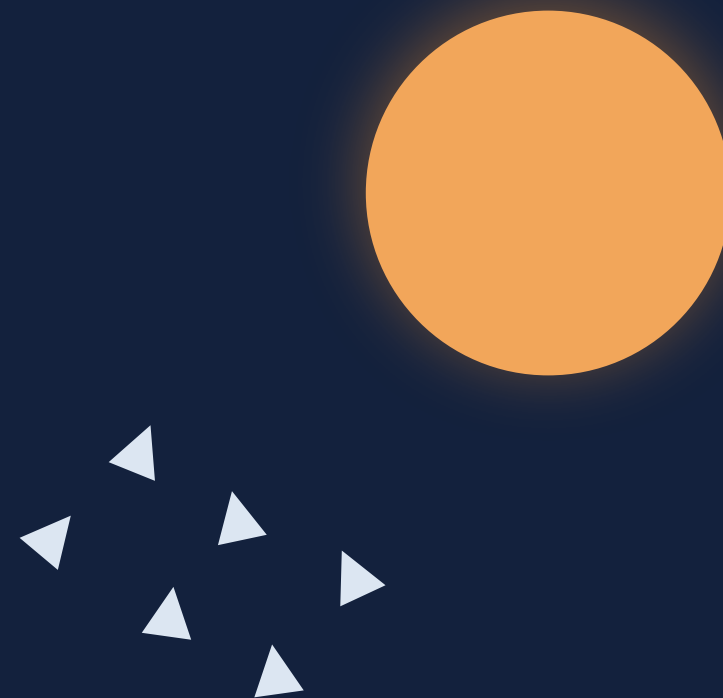


COMPUTER GRAPHICS: EXAM PROJECT

Boids_CG

Real-time GPGPU flocking simulation with Three.js and GLSL shaders

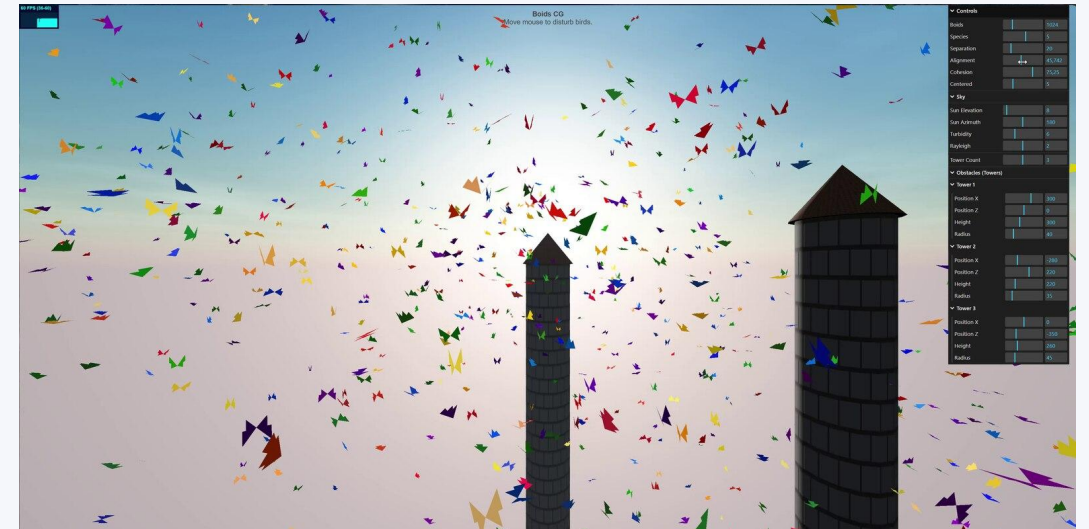


Francesco Bonaiuti

Repository: github.com/LytTheBit/Boids_CG

The project in brief

- Real-time flocking simulation: thousands of butterfly-shaped agents flying in a flock
- Collective behavior generated by simple local rules applied to each individual agent
- All physics, position, and velocity calculations for each boid are performed on the GPU using GLSL shaders (GPGPU technique) rather than the CPU
- Complete scene: procedural sky with sun, towers with obstacle avoidance, a mouse-controlled predator, and a control panel with real-time adjustable parameters
- Presentation objective: to demonstrate not only the result but also the computer graphics theory behind every implementation choice



Flock with procedural sky and towers (screenshot from the project)

Resources used

Bibliography and theory

- Computer Graphics course material and exercises
- S. Gortler, Foundations of 3D Computer Graphics
- J. Hughes et al., Computer Graphics: Principles and Practice
- C. Reynolds, Flocks, Herds and Schools: A Distributed Behavioral Model, SIGGRAPH 1987
- Preetham et al., A Practical Analytic Model for Daylight, SIGGRAPH 1999

Libraries and documentation

- Three.js: WebGL rendering engine
- GPUComputationRenderer: official Three.js add-on for texture-based computation
- Sky: Three.js add-on based on the Preetham model
- lil-gui: interactive control panel
- Vite: dev server and bundler

Tools and software architecture

1 **Three.js** Scene, camera, standard materials, lights, WebGL canvas management

2 **GPUComputationRenderer** GPGPU simulation on textures: it encapsulates the multi-pass technique we will look at shortly.

3 **GLSL** 4 custom shaders: 2 simulation and 2 rendering for bodies

4 **No external assets** Boid geometry and tower textures generated entirely at runtime.

In this presentation, we focus primarily on the computer graphics theory applied in the project: GPGPU-based rasterization, geometric transformations, lighting, and texture mapping.

What is flocking?

- A model proposed by Craig Reynolds in 1987 to simulate the flight of flocks, schools of fish, and herds.
- Core idea: no agent knows the global shape of the flock; each "boid" observes only its neighbors within a limited radius.
- The collective behavior we observe (the flock turning, bunching together, avoiding obstacles) is an emergent phenomenon: it arises from the sum of very simple individual rules.
- Three classic local rules applied to each boid at every frame:

1 Separation

moving away from neighbors that are too close, to avoid collisions

2 Alignment

adjust one's speed to match that of nearby vehicles

3 Cohesion

moving towards the center of mass of the Local Group

Why calculate flocking on the GPU?

- Applying the three rules requires each boid to compare its position with that of all others, incurring an $O(n^2)$ cost per frame.
- The course introduces shaders and GPUs specifically to leverage data parallelism: vertices and pixels are processed independently and simultaneously (SIMD).
- The project concept: treat each boid as an independent pixel and execute the same fragment shader in parallel across thousands of boids simultaneously.
- No data is ever sent back to the CPU during the simulation; state and updates remain entirely within textures on the GPU.



COURSE THEORY

"[...] computation to determine the color of one screen pixel can be performed completely independently from that for the other pixels."

In the project, a "pixel" is not a color: it is the STATE of a boid (position or velocity).

Same principle as the course, new meaning for the pixel: the agent's physical state.

Theory: The Framebuffer Object and the Multipass Technique

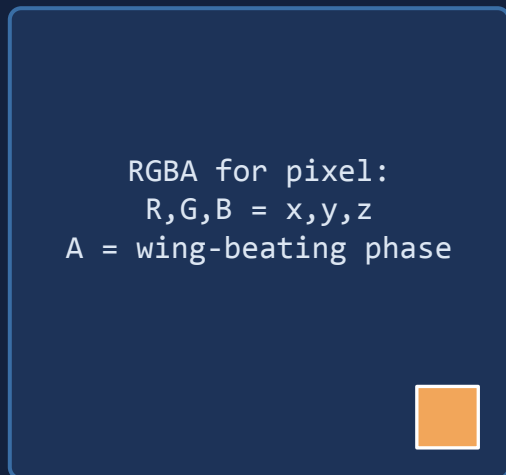
$f(x)$ HOW TO BUILD AN FBO

1. Create the framebuffer object
2. Create a texture; define its dimensions and parameters
3. Attach the texture to the framebuffer's color attachment
4. Check the framebuffer's status
5. Draw "into" the texture instead of onto the screen

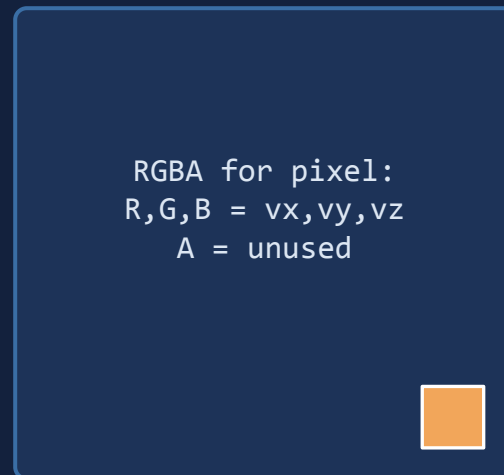
- A Framebuffer Object (FBO) allows rendering to a texture instead of the screen; this texture can then be read back as input for another rendering pass.
- GPUComputationRenderer applies this exact concept: it creates an FBO and a texture for each simulated variable (e.g., position, velocity).
- Ping-pong technique: frame N reads the textures produced in frame N-1 and writes new ones, which then serve as input for frame N+1.
- This follows the same pattern used in the course where a scene is rendered into a texture and subsequently reused as input for a second pass.

The boid as a texture pixel

texturePosition



textureVelocity



same pixel, same boid

```
boidCount = 1024 textureWidth = ceil( root(boidCount) ) = 32
texture 32x32
```

```
GL fragmentShaderVelocity.glsl GLSL

getBirdIndex(uv):
    pixel = floor(uv * resolution.xy)
    return pixel.y * resolution.x
        + pixel.x

getSpecies(birdIndex):
    return floor(birdIndex
        * speciesCount / boidCount)
```

- The boid's integer index is derived from the pixel's UV coordinates.
- The species to which it belongs is derived from the index and used for multi-species cohesion.

Theory: concentric zones and blending function

- Three increasing radii are defined around each boid, one for each rule: separation, alignment, and cohesion.
- The total radius of the interaction zone is the sum of the three radii.
- For each neighbor, the percentage of its distance falling within each of the three zones is calculated.
- The transition between zones is not abrupt but follows a cosine curve; this prevents sudden changes in speed when a neighbor enters or leaves a zone.

$f(x)$

KEY FORMULAS

$$\text{zoneRadius} = r\text{Sep} + r\text{Align} + r\text{Cohes}$$

$$\text{percent} = \text{distance}^2 / \text{zoneRadius}^2$$

$$\text{blend}(p) = 0.5 - \cos(p * 2\pi) * 0.5$$

$$f(\text{neighbor}) = \text{blend}(\text{percent}) * \text{delta}$$

The same concept of smooth transition frequently appears in computer graphics, where it prevents jerky behavior within the flock.

Code: separation, alignment, cohesion

```
GL fragmentShaderVelocity.glsl GLSL

zoneRadius = separationDistance
    + alignmentDistance + cohesionDistance;
percent = distSquared / zoneRadiusSquared;

if (percent < separationThresh) {
    // separazione: allontana i boid vicini
    f = (separationThresh/percent - 1.0)*delta;
    velocity -= normalize(dir) * f;

} else if (percent < alignmentThresh) {
    // allineamento alla velocita dei vicini
    f = (0.5 - cos(adjustedPercent*PI_2)
        * 0.5 + 0.5) * delta;
    velocity += normalize(birdVelocity) * f;

} else {
    // coesione verso il centro del gruppo
    velocity += normalize(dir) * f;
}
```

- Each boid compares its position with that of the others by reading `texturePosition` pixel by pixel.
- The `if` / `else if` structure applies only one rule at a time, based on the distance range into which the neighbor in question falls.
- The final velocity result is the sum of all these small contributions—one for each neighbor examined.

Code: species, obstacles, and recall to the center

```
GL fragmentShaderVelocity.glsl GLSL

// coesione solo tra boid della stessa specie
if (sameSpecies) {
    velocity += normalize(dir) * f;
}

// torre come capsula verticale
obstacleT = clamp(dot(selfPosition
    - obstacleBase, obstacleSegment)
    / obstacleSegLenSq, 0.0, 1.0);
obstacleDist = length(selfPosition
    - (obstacleBase
        + obstacleSegment * obstacleT));

if (obstacleDist < obstacleAvoidRadius) {
    velocity += normalize(obstacleDir) * f;
}
```

```
GL recall to the center GLSL

pullStrength = centerPull *
    smoothstep(0.0, UPPER_BOUNDS, dist);
velocity -= normalize(dir)
    * delta * pullStrength;
```

- Separation and alignment remain valid across different species: no flock physically passes through another.
- Cohesion, however, operates only within the same species: each group remains compact.
- Towers are modeled as capsules—a projection of the boid onto the base-to-top segment, with the minimum distance calculated based on the nearest point.
- Beyond the scene boundaries, a restorative force (smoothstep) gently steers the boids back toward the center.

Code: supplementation of the position

```
GL fragmentShaderPosition.glsl GLSL

vec3 position = texture2D(
    texturePosition, uv).xyz;
vec3 velocity = texture2D(
    textureVelocity, uv).xyz;

float phase = tmpPos.w;
phase = mod(phase + delta
    + length(velocity.xz)*delta*3.0
    + max(velocity.y, 0.0)*delta*6.0,
    62.83);

gl_FragColor = vec4(
    position + velocity * delta * 15.0,
    phase
);
```

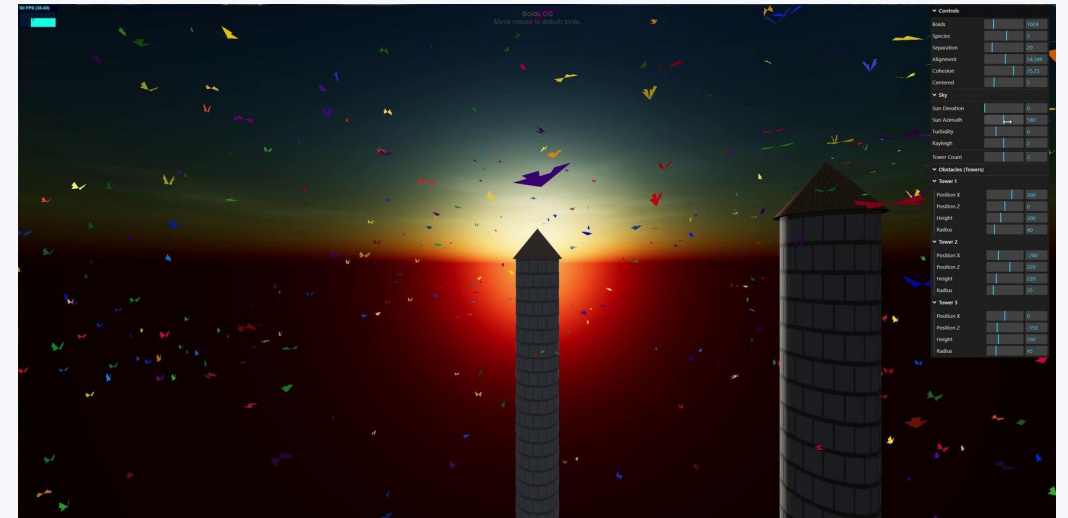
$f(x)$ Euler integration

$$p(t + dt) = p(t) + v(t) * dt$$

- Explicit integration: the new position is the old position plus velocity multiplied by the time elapsed since the previous frame (delta).
- The w-component of the position texture is not spatial; it is reused to advance the wing-flapping phase—speeding up when the boid is flying fast horizontally or ascending.

Practical observations and result

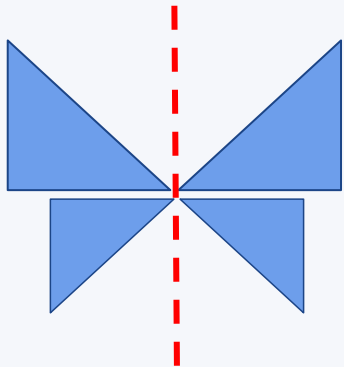
- If the three vectors (separation, alignment, cohesion) are poorly balanced, the flock may exhibit jitter: the boids oscillate back and forth instead of stabilizing.
- With a very large delta (low frame rate), a boid might pass through a tower in a single integration step, bypassing collision detection; this is a typical limitation of fixed-step explicit integration.
- If the obstacle avoidance radius is too small, boids may get trapped against the tower's surface instead of navigating around it.
- These trade-offs are resolved empirically by adjusting the parameters via the control panel.



Result: multi-species flock in flight around the towers

The starting mesh: the butterfly

- Each boid is composed of 4 triangles (upper, lower, left, and right wings), with no separate body.
- All 4 triangles share the same pivot vertex at (0,0,z); this is a necessary condition for rigidly folding the wings within the vertex shader.
- Custom `reference` attribute: UV coordinates pointing to the pixel in the GPGPU texture corresponding to that boid.
- `birdColor` and `birdScale` attributes: species-specific color and individual scale, purely aesthetic, with no effect on physics.



```
JS BirdGeometry.js JS
const x = ((birdIndex % textureWidth)
  + 0.5) / textureWidth;
const y = (Math.floor(birdIndex
  / textureWidth) + 0.5) / textureWidth;

references.array[v*2+0] = x;
references.array[v*2+1] = y;

// nel vertex shader:
// texture2D(texturePosition, reference)
```

The `reference` attribute is the key that links a mesh vertex to its state calculated via GPGPU, serving the same role as the attributes and texture coordinates covered in the course.

Theory: affine transformations and elementary rotations

f(x)

ROTATIONS AROUND THE AXES

Ry(theta) =

$$\begin{vmatrix} \cos(t) & 0 & \sin(t) \\ 0 & 1 & 0 \\ -\sin(t) & 0 & \cos(t) \end{vmatrix}$$

Rz(phi) =

$$\begin{vmatrix} \cos(p) & -\sin(p) & 0 \\ \sin(p) & \cos(p) & 0 \\ 0 & 0 & 1 \end{vmatrix}$$

- The course demonstrates how any 3D rotation can be decomposed into elementary rotations around the Cartesian axes.
- The order of transformations matters: matrix composition is non-commutative.
- In the project, we apply this principle twice: first, the boid is scaled; then, the wings are folded; finally, it is oriented along the direction of flight.
- The vertex shader constructs the Ry and Rz matrices shown here, starting solely from the velocity vector and without ever computing the inverse of cosine or sine.

Code: rigid wing folding

```
GL birdVertex.glsl GLSL

vec3 scaledPosition = position * birdScale;

float maxFoldAngle = radians(75.0);
float foldAngle = maxFoldAngle *
    (0.5 + 0.5 * sin(tmpPos.w));

float foldSin = sin(foldAngle);
float foldCos = cos(foldAngle);

// bends around the body axis (z);
// the hinge (x=0) remains stationary
newPosition.y = abs(scaledPosition.x)
    * foldSin;
newPosition.x = scaledPosition.x * foldCos;
```

- 2D rotation in the (x, y) plane around the z-axis, applied per vertex.
- First, scaling (birdScale) is applied, then folding: this way, the hinge remains at (0, 0, z) and the scaling does not shift it. If the order were reversed, the fold point would shift along with the scaling.
- Using abs(x) causes the left and right wings to fold in the same direction, like closing a book, rather than symmetrically.

Code: orientation along the flight direction

```
GL birdVertex.glsl GLSL

float xz = length(velocity.xz);
float cosry = velocity.x / xz;
float sinry = velocity.z / xz;
float cosrz = sqrt(1.0 - velocity.y*velocity.y);
float sinrz = velocity.y;

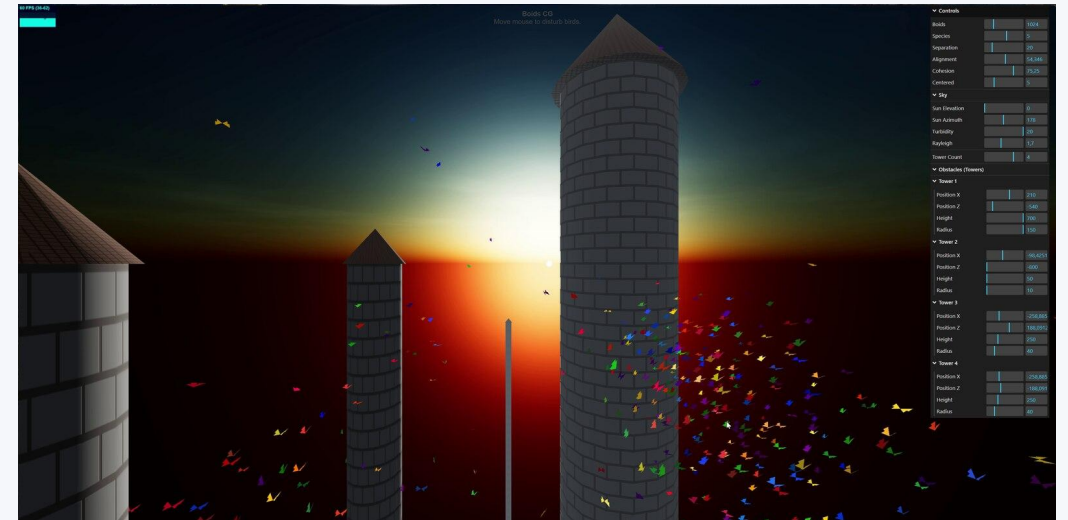
mat3 maty = mat3(cosry, 0, -sinry,
                 0, 1, 0,
                 sinry, 0, cosry);
mat3 matz = mat3(cosrz, sinrz, 0,
                 -sinrz, cosrz, 0,
                 0, 0, 1);

newPosition = maty * matz * newPosition;
```

- cos and sin are derived directly from the normalized velocity components, without calling inverse trigonometric functions, which is more efficient on the GPU.
- maty and matz are exactly the Ry and Rz matrices seen in the theory: two elementary rotations combined.
- The result: the butterfly is always oriented so that flying forward aligns with the actual velocity direction.

A limiting case and the final result

- When the boid flies almost vertically, xz (the length of the velocity projected onto the horizontal plane) approaches zero.
- $\cos\theta$ and $\sin\theta$ become a $0/0$ ratio; the horizontal direction is no longer defined.
- The course highlights precisely this type of unnatural behavior when representing rotations using angles rather than quaternions, which is one of the reasons why the latter are preferred for interpolating general rotations.
- In the project, this case is rare and transient (the boid never remains stationary in a vertical position), so the visual artifact is minimal.



Result: butterflies with dynamic folding and orientation.

Theory: local lighting and linear fog

$f(x)$ LAMBERT'S COSINE LAW

$\text{intensity} = \text{color} * \max(0, N \cdot L)$

N = surface normal

L = direction towards the light

$f(x)$ LINEAR FOG

$\text{fogFactor} = (\text{end} - \text{distance})$
 $\quad \quad \quad / (\text{end} - \text{start})$

$\text{color} = \text{surface} * \text{fogFactor}$
 $\quad \quad + \text{fogColor} * (1 - \text{fogFactor})$

- Lambert's law describes diffuse materials; the towers use a standard Three.js material that automatically applies this law alongside an ambient component.
- Linear fog attenuates color based on the distance from the viewer.
- The boids' fragment shader applies a simplified version of the same principle: it directly attenuates brightness based on depth, without generating a fully lit shader.

Code: boid fragment shader

```
birdFragment.glsl

varying vec4 vColor;
varying float z;

void main() {
    float depthFactor =
        0.4 + (1000.0 - z) / 1000.0 * 0.6;
    depthFactor = clamp(depthFactor,
        0.25, 1.0);

    vec3 finalColor = vColor.rgb
        * depthFactor;
    gl_FragColor = vec4(finalColor, 1.0);
}
```

- Unlit shader: the color arrives ready-made from the vertex shader via the `vColor` varying, consistent with the vertex-to-fragment interpolation covered in the course.
- z (depth in world space) linearly attenuates brightness based on distance, echoing the concept of linear fog demonstrated earlier.
- Effect: distant boids appear darker, reinforcing the perception of depth even without a true lighting model.

Code: procedural sky, lights, and fog

```
index.js JS
sky.material.uniforms.turbidity.value = 6;
sky.material.uniforms.rayleigh.value = 1.2;

sun.setFromSphericalCoords(1,
    phiFromElevation, thetaFromAzimuth);
sky.material.uniforms.sunPosition.value
    = sun;

directionalLight.position.copy(sun);
hemisphereLight.intensity = 0.35;

scene.fog = new THREE.Fog(
    skyColor, near, far);
```

- The procedural sky follows the Preetham model: a physically-based atmospheric scattering approach parameterized by turbidity and the Rayleigh coefficient.
- The sun's direction drives the scene's directional light, ensuring that shadows and the warm tones of the towers align with the set time of day.
- The hemispherical light provides soft fill lighting, preventing shadowed areas from appearing completely black.

Code: procedural textures via Canvas 2D

```
JS index.js JS
const canvas = document.createElement('canvas');
ctx.fillStyle = '#9c9285';
ctx.fillRect(0, 0, canvas.width, canvas.height);

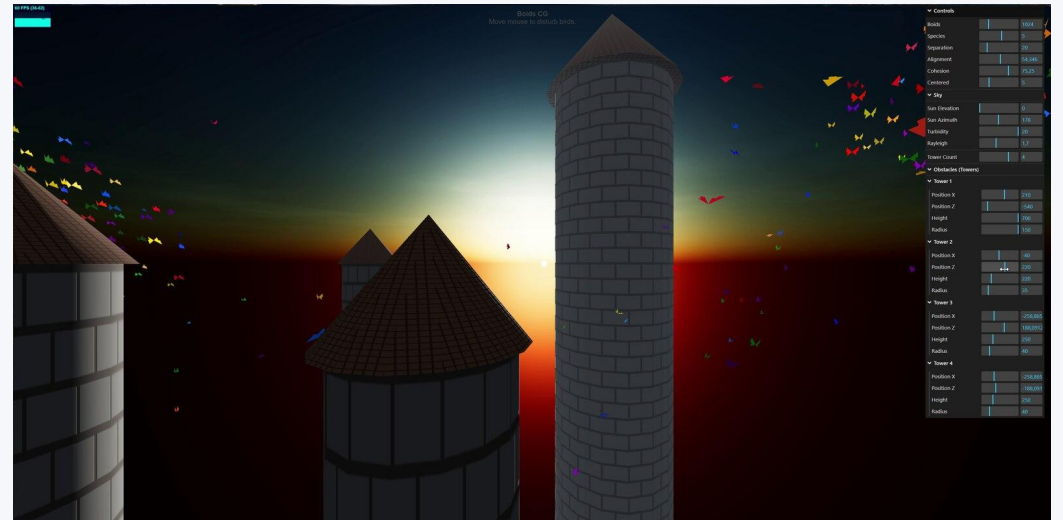
for (let y = 0; y < canvas.height; y +=
  brickHeight) {
  ctx.fillStyle = '#55493f';
  ctx.fillRect(0, y, canvas.width,
    mortarThickness);
}

towerBodyTexture = new THREE.CanvasTexture(canvas);
towerBodyTexture.wrapS = THREE.RepeatWrapping;
towerBodyTexture.repeat.set(4, 8);
```

- No external images: the brick texture is drawn at runtime onto a 2D canvas and loaded as a `THREE.CanvasTexture`.
- `repeat.set()` applies tiling: the same small texture is repeated multiple times across the tower's cylindrical surface.
- It is the same concept of texture coordinates and wrapping covered in the course, applied to a generated texture rather than one loaded from a file.

Typical artifacts and final result

- Aliasing: the triangular wing edges, being very small on screen, may exhibit the classic "jaggies" (stair-step effect) described earlier regarding discrete image sampling.
- Flickering: when a boid is distant and its wings cover less than a single pixel, the sampled color can fluctuate unstably between frames, precisely the phenomenon where small objects appear to flicker.
- Visible tiling: if the repeat value is too low relative to the tower's size, the repetition of the procedural texture becomes noticeable.
- No lighting model on boids: as this is an unlit shader, the appearance remains intentionally flat compared to the towers, which utilize a physically-based material.



Result: procedural sky, fog, and illuminated towers.

Parameters: compile-time and runtime

```

JS simulation.js / index.js JS
this.velocityVariable.material.defines
    .MAX_OBSTACLES = this.obstacleCount.toFixed(0);

function rebuildSimulation() {
    if (birdMesh) { scene.remove(birdMesh); }
    if (simulation) { simulation.dispose(); }
    simulation = new BoidsSimulation(...);
    initBirds();
}

```

- In GLSL, a uniform array requires a fixed size known at shader compile time.
- `MAX\_OBSTACLES` (the number of towers) is defined via `#define`; changing it from the GUI requires rebuilding the entire simulation.
- Parameters such as the distances for the three rules or the centering force are uniforms, meaning they can be modified in real time without recompilation.
- The lil-gui panel exposes both categories of parameters, calling `rebuildSimulation()` only when necessary.

Performance analysis

- Unlike a comparison between alternative techniques, here there is only one pipeline; we therefore evaluate how performance scales as the load varies.
- The simulation cost depends on the GPGPU texture resolution (`boidCount`), not on the number of triangles drawn.
- The rendering cost, on the other hand, depends on the number of triangles, equal to four times the `boidCount`, regardless of the number of species.

Number of boids	Species	Obstacles	Average FPS	Note
500	4	3	60fps	reference
2 000	4	3	60fps	
4 000	4	3	60fps	
4 000	8	3	60fps	
4 000	8	5	60fps	heavy load

My PC features an AMD Ryzen 5 7600X CPU, 32 GB of RAM, and an NVIDIA GeForce RTX 5070 with 12 GB of VRAM, providing strong performance for Computer Graphics simulations.

$f(x)$ Why does it scale like that?

The code is well-optimized, and it maintains a steady 60 fps even during the most hectic moments.

What the project demonstrates

- Practical application of the GPGPU concept: utilizing the graphics pipeline, specifically fragment shaders operating on textures, for general-purpose computation rather than merely color generation.
- Combined use of multiple course concepts: attributes, uniforms, and varyings; affine transformations and elementary rotations; texture mapping; diffuse and ambient lighting; linear fog; and sampling and aliasing.
- Complex emergent behavior, specifically multi-species flocks avoiding obstacles, achieved through simple, fully parallelized local rules.

Possibili estensioni

- Truly dynamic number of towers, using uniform buffers instead of #define
- Shadows cast by boids and towers via shadow mapping
- Occlusion culling for much larger flocks
- Dedicated anti-aliasing to reduce flickering on distant wings
- Physically accurate lighting, including for boids

Thank you for your attention.